



Chrome notifications get quick reply

According to the Chromium Gerrit, websites will be able to send you a notification with added Quick Reply support. <https://dgit.in/Aug18-51-1>



Who wins with PWAs?

Microsoft joined with Google to push Progressive Web Apps together, but Google might benefit the most out of it. <https://dgit.in/Aug18-51-2>

tech

DEVWORX

standard $z=wx+b$ operation on it. Also add a condition that allows the caller to add RELU to the layer as shown at <https://dgit.in/TF5>. Thus we have completed defining the architecture of our network!

Placeholders and input

Now, let's create a placeholder that will hold the input training images. All the input images are read in dataset.py file and resized to 128 x 128 x 3 size. Input placeholder x is created in the shape of [None, 128, 128, 3]. The first dimension being None means you can pass any number of images to it. Next, we pass images in the batch of 16 hence shape will be [16 128 128 3].

Similarly, we create a placeholder `y_true` for storing the predictions. For each image, we have four outputs i.e. probabilities for each class. Hence `y_pred` is of the shape [None 4] (for batch size 16 it will be [16 4] as shown:

```
x = tf.placeholder(tf.float32,
shape=[None, img_size,img_
size,num_channels], name='x')
```

```
y_true = tf.placeholder(tf.
float32, shape=[None, num_
classes], name='y_true')
```

```
y_true_cls = tf.argmax(y_true,
dimension=1)
```

NETWORK DESIGN

We create various layers of the network as shown at <https://dgit.in/TF7>.

Predictions

We can get the probability of each class by applying softmax to the output of the final fully connected layer.

```
y_pred = tf.nn.softmax(layer_
fc2,name='y_pred')
```

`y_pred` contains the predicted probability of each class for each image.

The class having higher probability is the prediction of the network. Let us now define the cost that will be minimized to reach the optimum value of variables. We will use a cost that will be calculated using the Tensorflow function:

```
cross_entropy = tf.nn.soft-
max_cross_entropy_with_
logits(logits=layer_fc2,
labels=y_true)
cost = tf.reduce_mean(cross_
entropy)
```

We will leave the details of the implementation for now.



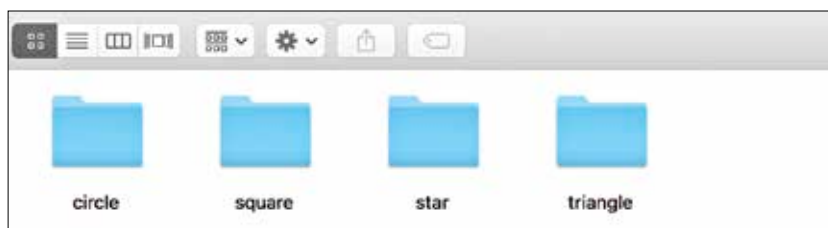
Optimization

There are a number of optimization functions in tensorflow, we will use the AdamOptimizer for gradient calculation and weight optimization. The learning rate is defined as the rate of change implemented by the optimizer, for our use case we will set it to 0.0001.

```
optimizer = tf.train.
AdamOptimizer(learning_
rate=1e-4).minimize(cost)
```

We next split the net training images into batches as follows:

```
x_batch, y_true_batch, _,
cls_batch = data.train.next_
```



*POINTERS



*First Steps with TensorFlow: Toolkit

Understanding the basics of Tensorflow as a part of Google's crash course on Machine Learning. It also comes with a video lecture and coding exercises. <https://dgit.in/TFToolkit>



*Tensorflow Tutorials using Jupyter Notebook

This series of TensorFlow tutorials is written in Python with Jupyter Notebook. These tutorials are intended for TensorFlow beginners. <https://dgit.in/TFJuPy>



*Build Your First Tensorflow Android App

This tutorial helps you getting started with bringing your Tensorflow models into your Android applications and requires Python, Java knowledge as well. <https://dgit.in/TFAndroid>